```
"""
Install all required libraries for the sentiment analysis project.

transformers -> pretrained NLP models like BERT, DistilBERT
datasets -> ready-to-use NLP datasets
torch -> deep learning framework used to train the model
scikit-learn -> evaluation metrics
pandas -> optional data handling
"""

!pip install transformers datasets torch scikit-learn pandas
```

```
Requirement already satisfied: torch in /usr/local/lib/python3.12/dist-packages (2.10.0+cpu)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (1.6.1)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (2.2.2)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from transformers) (3.25.0)
Requirement already satisfied: huggingface-hub<2.0,>=1.3.0 in /usr/local/lib/python3.12/dist-packages (from transformers)
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.12/dist-packages (from transformers) (2.0.2)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from transformers) (26.0)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from transformers) (6.0.3)
Requirement already satisfied: regex!=2019.12.17 in /usr/local/lib/python3.12/dist-packages (from transformers) (2025.11.3
Requirement already satisfied: tokenizers<=0.23.0,>=0.22.0 in /usr/local/lib/python3.12/dist-packages (from transformers)
Requirement already satisfied: typer-slim in /usr/local/lib/python3.12/dist-packages (from transformers) (0.24.0)
Requirement already satisfied: safetensors>=0.4.3 in /usr/local/lib/python3.12/dist-packages (from transformers) (0.7.0)
Requirement already satisfied: tqdm>=4.27 in /usr/local/lib/python3.12/dist-packages (from transformers) (4.67.3)
Requirement already satisfied: pyarrow>=15.0.0 in /usr/local/lib/python3.12/dist-packages (from datasets) (18.1.0)
Requirement already satisfied: dill<0.3.9,>=0.3.0 in /usr/local/lib/python3.12/dist-packages (from datasets) (0.3.8)
Requirement already satisfied: requests>=2.32.2 in /usr/local/lib/python3.12/dist-packages (from datasets) (2.32.4)
Requirement already satisfied: xxhash in /usr/local/lib/python3.12/dist-packages (from datasets) (3.6.0)
Requirement already satisfied: multiprocess<0.70.17 in /usr/local/lib/python3.12/dist-packages (from datasets) (0.70.16)
Requirement already satisfied: fsspec<=2025.3.0,>=2023.1.0 in /usr/local/lib/python3.12/dist-packages (from fsspec[http]<=
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.12/dist-packages (from torch) (4.15.0)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch) (75.2.0)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch) (3.6.1)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from torch) (3.1.6)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.16.3)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.5.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (3.6.0)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas) (2.9.0.post
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.3)
Requirement already satisfied: aiohttp!=4.0.0a0,!=4.0.0a1 in /usr/local/lib/python3.12/dist-packages (from fsspec[http]<=2
Requirement already satisfied: hf-xet<2.0.0,>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0,>
Requirement already satisfied: httpx<1,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0,>=1.3
Requirement already satisfied: typer in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0,>=1.3.0->transfo
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas) (
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests>=2.32.2-
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests>=2.32.2->datasets) (
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests>=2.32.2->datas
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests>=2.32.2->datas
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch) (
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2->torch) (3.0.3)
Requirement already satisfied: aiohappyeyeballs>=2.5.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!
Requirement already satisfied: aiosignal>=1.4.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4.0.0a
Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4.0.0a1->
Requirement already satisfied: frozenlist>=1.1.1 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4.0.0
Requirement already satisfied: multidict<7.0,>=4.5 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4.0
Requirement already satisfied: propcache>=0.2.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4.0.0a
Requirement already satisfied: yarl<2.0,>=1.17.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4.0.0
Requirement already satisfied: anyio in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface-hub<2.
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingfac
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages (from httpcore==1.*->httpx<1,>=0.23.0-
Requirement already satisfied: click>=8.2.1 in /usr/local/lib/python3.12/dist-packages (from typer->huggingface-hub<2.0,>=
Requirement already satisfied: shellingham>=1.3.0 in /usr/local/lib/python3.12/dist-packages (from typer->huggingface-hub<
Requirement already satisfied: rich>=12.3.0 in /usr/local/lib/python3.12/dist-packages (from typer->huggingface-hub<2.0,>=
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/python3.12/dist-packages (from typer->huggingface-hu
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from rich>=12.3.0->typer-
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from rich>=12.3.0->type
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-packages (from markdown-it-py>=2.2.0->rich>=12
```

```
"""
Import all required Python libraries.

torch -> deep learning framework
datasets -> to load the IMDB dataset
transformers -> tokenizers and pretrained models
numpy -> numerical operations
sklearn -> evaluation metrics
"""

import torch
import numpy as np

from datasets import load_dataset
```

```
from transformers import (
    AutoTokenizer,
    AutoModelForSequenceClassification,
    Trainer,
    TrainingArguments
)

from sklearn.metrics import accuracy_score, precision_recall_fscore_support
```

```
"""
Load the IMDB movie review dataset.

The dataset contains:
50,000 movie reviews labeled as:
0 -> Negative
1 -> Positive

The dataset automatically contains:
train split
test split
"""

dataset = load_dataset("imdb")

# Print dataset structure
print(dataset)
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
README.md:        7.81k/?  [00:00<00:00, 800kB/s]

plain_text/train-00000-of-00001.parquet: 100%                    21.0M/21.0M  [00:00<00:00, 105MB/s]

Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and fast
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN t
plain_text/test-00000-of-00001.parquet: 100%                    20.5M/20.5M  [00:00<00:00, 102MB/s]

plain_text/unsupervised-00000-of-00001.p(…): 100%                    42.0M/42.0M  [00:01<00:00, 68.5MB/s]

Generating train split: 100%                    25000/25000  [00:00<00:00, 95742.81 examples/s]

Generating test split: 100%                    25000/25000  [00:00<00:00, 129228.72 examples/s]

Generating unsupervised split: 100%                    50000/50000  [00:00<00:00, 178411.12 examples/s]

DatasetDict({
    train: Dataset({
        features: ['text', 'label'],
        num_rows: 25000
    })
    test: Dataset({
        features: ['text', 'label'],
        num_rows: 25000
    })
    unsupervised: Dataset({
        features: ['text', 'label'],
        num_rows: 50000
    })
})
```

```
"""
Load the tokenizer for DistilBERT.

Tokenization converts text into tokens (numbers)
that the transformer model can understand.
"""

tokenizer = AutoTokenizer.from_pretrained("distilbert-base-uncased")
```

```
config.json: 100%                    483/483  [00:00<00:00, 18.7kB/s]

tokenizer_config.json: 100%                    48.0/48.0  [00:00<00:00, 4.45kB/s]

vocab.txt:        232k/?  [00:00<00:00, 12.9MB/s]

tokenizer.json:        466k/?  [00:00<00:00, 24.0MB/s]
```

```
"""
Tokenize the input text.
```

```
    Parameters
    ----------
    example : dict
        A single dataset example containing the review text.

    Returns
    -------
    dict
        Tokenized text including:
        input_ids
        attention_mask
    """

    def tokenize_function(example):

        # Convert raw text into token IDs
        return tokenizer(
            example["text"],          # the review text
            truncation=True,          # cut long reviews
            padding="max_length",     # pad shorter reviews
            max_length=256            # maximum sequence length
        )
```

```
    """
    Apply tokenization to the entire dataset.

    map() processes every example in the dataset
    and adds tokenized features.
    """

    tokenized_dataset = dataset.map(tokenize_function, batched=True)

    print(tokenized_dataset)
```

```
Map: 100%                                          25000/25000 [00:17<00:00, 1388.47 examples/s]

Map: 100%                                          25000/25000 [00:18<00:00, 1414.88 examples/s]

Map: 100%                                          50000/50000 [00:35<00:00, 1261.77 examples/s]
DatasetDict({
    train: Dataset({
        features: ['text', 'label', 'input_ids', 'token_type_ids', 'attention_mask'],
        num_rows: 25000
    })
    test: Dataset({
        features: ['text', 'label', 'input_ids', 'token_type_ids', 'attention_mask'],
        num_rows: 25000
    })
    unsupervised: Dataset({
        features: ['text', 'label', 'input_ids', 'token_type_ref', 'attention_mask'],
        num_rows: 50000
    })
})
```

```
    """
    Split dataset into training and testing sets.

    train_dataset -> used to train the model
    test_dataset -> used to evaluate the model
    """

    train_dataset = tokenized_dataset["train"]
    test_dataset = tokenized_dataset["test"]
```

```
    """
    Load the pretrained DistilBERT model for sequence classification.

    num_labels = 2
    because sentiment classification has:
    0 -> Negative
    1 -> Positive
    """

    model = AutoModelForSequenceClassification.from_pretrained(
        "distilbert-base-uncased",
        num_labels=2
    )
```

```
model.safetensors: 100%                                                    268M/268M [00:03<00:00, 318MB/s]

Loading weights: 100%                          100/100 [00:00<00:00, 436.87it/s, Materializing param=distilbert.transformer.layer.5.sa_layer_norm.weight]
DistilBertForSequenceClassification LOAD REPORT from: distilbert-base-uncased
Key                     | Status     |
------------------------+------------+-
vocab_projector.bias    | UNEXPECTED |
vocab_layer_norm.bias   | UNEXPECTED |
vocab_layer_norm.weight | UNEXPECTED |
vocab_transform.bias    | UNEXPECTED |
vocab_transform.weight  | UNEXPECTED |
classifier.weight       | MISSING    |
classifier.bias         | MISSING    |
pre_classifier.bias     | MISSING    |
pre_classifier.weight   | MISSING    |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING   :those params were newly initialized because missing from the checkpoint. Consider training on your downstream t
```

```
"""
Compute evaluation metrics for the model.

Metrics calculated:
Accuracy
Precision
Recall
F1-score
"""

def compute_metrics(pred):

    # True labels
    labels = pred.label_ids

    # Predicted labels
    preds = np.argmax(pred.predictions, axis=1)

    # Compute precision, recall, f1 score
    precision, recall, f1, _ = precision_recall_fscore_support(
        labels,
        preds,
        average='binary'
    )

    # Compute accuracy
    acc = accuracy_score(labels, preds)

    return {
        "accuracy": acc,
        "precision": precision,
        "recall": recall,
        "f1": f1
    }
```

```
training_args = TrainingArguments(
    output_dir="./results",
    eval_strategy="epoch",  # evaluate after each epoch
    learning_rate=2e-5,
    per_device_train_batch_size=8,
    per_device_eval_batch_size=8,
    num_train_epochs=2,
    weight_decay=0.01,
    logging_dir="./logs"
)
```

```
`logging_dir` is deprecated and will be removed in v5.2. Please set `TENSORBOARD_LOGGING_DIR` instead.
```

```
"""
Trainer is a Hugging Face utility that handles:
training
evaluation
logging
checkpointing
"""

trainer = Trainer(

    model=model,

    args=training_args,
```

```
    train_dataset=train_dataset,

    eval_dataset=test_dataset,

    compute_metrics=compute_metrics
)
```

```
"""
Start training the transformer model.

The training process will show:
training loss
evaluation loss
metrics
"""

trainer.train()
```

```
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:775: UserWarning: 'pin_memory' argument is set as tru
  super().__init__(loader)
```

[295/6250 39:44 < 13:27:42, 0.12 it/s, Epoch 0.09/2]

Epoch	Training Loss	Validation Loss

[438/6250 59:04 < 13:07:32, 0.12 it/s, Epoch 0.14/2]

Epoch	Training Loss	Validation Loss

```
"""
Test the trained model on a custom review.

This demonstrates how the model predicts sentiment
for unseen text.
"""

# Example review
text = "This movie was absolutely fantastic and very enjoyable."

# Tokenize the text
inputs = tokenizer(text, return_tensors="pt", truncation=True, padding=True)

# Get model predictions
outputs = model(**inputs)

# Get predicted class
predicted_class = torch.argmax(outputs.logits).item()

# Convert label to sentiment
if predicted_class == 1:
    print("Sentiment: Positive 😊")
else:
    print("Sentiment: Negative 😞")
```